

Day 35 = Stack + Queue Practice (Mixed Problems)

Day 35 – Stack & Queue Practice

1. Stack Practice Problems

Problem 1: Next Greater Element (Revision)

Concept: Stack use karke right side ka next greater element find karte hain

 Already Day 31 me kiya, aaj revise + optimize

Problem 2: Stock Span Problem

Question:

Har din ke liye batao ki pichle kitne din tak stock price usse chhota ya equal tha

 Approach:

Stack me indices store karo

Jab tak current price > stack top → pop

Java Code:

```
1. import java.util.*;

class StockSpan {
    public static void calculateSpan(int price[], int n) {
        Stack<Integer> stack = new Stack<>();
        int span[] = new int[n];

        stack.push(0);
        span[0] = 1;

        for (int i = 1; i < n; i++) {
            while (!stack.isEmpty() && price[i] >= price[stack.peek()]) {
                stack.pop();
            }

            span[i] = stack.isEmpty() ? (i + 1) : (i - stack.peek());
            stack.push(i);
        }

        for (int s : span) {
            System.out.print(s + " ");
        }
    }

    public static void main(String[] args) {
        int price[] = {100, 80, 60, 70, 60, 75, 85};
        calculateSpan(price, price.length);
    }
}
```

✅ Problem 3: Min Stack (Important 🔥)

Question: Stack me minimum element $O(1)$ me kaise nikale?

💡 Idea:

2 stacks use karo:

Normal stack

Min stack

✅ Java Code:

```
1. import java.util.*;

class MinStack {
    Stack<Integer> mainStack = new Stack<>();
    Stack<Integer> minStack = new Stack<>();

    void push(int x) {
        mainStack.push(x);

        if (minStack.isEmpty() || x <= minStack.peek()) {
            minStack.push(x);
        }
    }

    void pop() {
        if (mainStack.peek().equals(minStack.peek())) {
            minStack.pop();
        }
        mainStack.pop();
    }

    int getMin() {
        return minStack.peek();
    }
}
2.
```

2. Queue Practice Problems

Problem 4: First Negative Number in Window (Sliding Window + Queue)

Question:

Har window (size k) me first negative number find karo

 Approach:

Queue me negative elements store karo

Window slide karte raho

Java Code:

```
1. import java.util.*;

class FirstNegative {
    public static void printFirstNegative(int arr[], int k) {
        Queue<Integer> q = new LinkedList<>();

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] < 0) {
                q.add(i);
            }

            if (i >= k - 1) {
                while (!q.isEmpty() && q.peek() < i - k + 1) {
                    q.poll();
                }

                if (q.isEmpty()) {
                    System.out.print("0 ");
                } else {
                    System.out.print(arr[q.peek()] + " ");
                }
            }
        }

        public static void main(String[] args) {
            int arr[] = {12, -1, -7, 8, -15, 30, 16, 28};
            printFirstNegative(arr, 3);
        }
    }
}
```

✅ Problem 5: Circular Queue (Revision)

👉 Already Day 33 me kiya

Practice:

Enqueue

Dequeue

Full / Empty condition

✅ Problem 6: Queue using Stack 🔥

Concept:

Queue ko stack se implement karna

💡 Approach:

2 stacks use karo

stack1 → push

stack2 → pop

✅ Java Code:

```
1. import java.util.*;

class QueueUsingStack {
    Stack<Integer> s1 = new Stack<>();
    Stack<Integer> s2 = new Stack<>();

    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if (s2.isEmpty()) {
            while (!s1.isEmpty()) {
                s2.push(s1.pop());
            }
        }
        return s2.pop();
    }
}

2.
```